Data Science: From Fundamentals to Advanced

Authored by: Siddharth Vidyarthi

Table of Contents

Introduction

Data Science is the interdisciplinary field that combines mathematics, statistics, programming, and domain expertise to extract actionable insights from data. In 2025, 90% of global data has been created in the last two years alone, making data science critical for competitive advantage[1]. This comprehensive guide takes you from foundational concepts through advanced techniques deployed by FAANG companies and global leaders.

Why Data Science Matters

- **Business Impact**: 64% of organizations investing in big data analytics report competitive advantage[1]
- **Career Growth**: Demand for data scientists grew 36% annually (2015-2024), outpacing overall job growth[1]
- **Innovation Driver**: Companies like Amazon, Netflix, Uber, and Google generate billions in value through data science[2]
- **Transformative Power**: From personalized recommendations to fraud detection to drug discovery

Data Science Workflow

1. **Problem Definition**: Frame business question
2. **Data Collection**: Gather relevant data
3. **Data Cleaning**: Handle missing values, duplicates, inconsistencies
4. **Exploratory Analysis**: Understand patterns and relationships
5. **Feature Engineering**: Create meaningful predictors
6. **Model Selection**: Choose appropriate algorithms
7. **Model Training & Validation**: Build and evaluate
8. **Hyperparameter Tuning**: Optimize performance
9. **Deployment**: Operationalize in production
10. **Monitoring**: Track performance over time

Module 1: Foundations & Programming

1.1 Fundamentals of Data Science

Core Concepts:

- **Data**: Raw facts and observations
- **Information**: Processed data with context
- **Knowledge**: Understanding relationships and insights
- **Wisdom**: Actionable intelligence from knowledge

Data Types:

- **Structured**: Tables, databases, CSV (rows × columns)
- **Unstructured**: Text, images, audio, video
- **Semi-structured**: JSON, XML (nested formats)

Problem Types:

- **Classification**: Predict categories (spam/not spam, fraud/not fraud)
- **Regression**: Predict continuous values (price, temperature, sales)
- **Clustering**: Group similar items (customer segmentation)
- **Anomaly Detection**: Identify outliers (fraud detection)
- **Recommendation**: Suggest items (Netflix recommendations)
- **Forecasting**: Predict future values (stock prices, demand)

1.2 Python for Data Science

Essential Libraries:

NumPy: Numerical computing

- Arrays and matrices
- Mathematical operations
- Linear algebra

Pandas: Data manipulation

- DataFrames and Series
- Data cleaning and transformation
- Aggregation and grouping

Matplotlib & Seaborn: Visualization

- Static plots
- Interactive visualizations
- Statistical graphics

Scikit-learn: Machine learning

- Model building
- Cross-validation
- Preprocessing

TensorFlow/PyTorch: Deep learning

- Neural networks
- GPU acceleration

- Production deployment

1.3 Key Python Concepts

Data Structures:

- Lists, Tuples, Sets, Dictionaries
- Performance considerations
- Use cases for each

Functions & Modules:

- Function definition and scope
- Lambda functions
- Module import and organization

Object-Oriented Programming:

- Classes and objects
- Inheritance and polymorphism
- Essential for production code

Error Handling:

- Try-except blocks
- Logging and debugging
- Writing robust code

Module 2: Data Processing & Statistics

2.1 Data Cleaning & Preprocessing

Handling Missing Data:

- Detection: .isnull(), .info()
- Removal: Delete rows/columns with missing values
- Imputation: Mean, median, mode, forward/backward fill
- Advanced: KNN imputation, MICE, prediction-based

Outlier Detection & Treatment:

- Z-score method: Values beyond ±3 standard deviations

- IQR method: Values outside 1.5 × IQR from quartiles
- Isolation Forest: Unsupervised anomaly detection
- Treatment: Removal, transformation, or separate analysis

Data Scaling & Normalization:

- Standardization: (X - mean) / std dev
- Normalization: (X - min) / (max - min)
- When to use: Required for distance-based algorithms

Encoding Categorical Variables:

- Label Encoding: Ordered categories → numeric
- One-Hot Encoding: Categorical → binary columns
- Target Encoding: Category → mean target value

2.2 Exploratory Data Analysis (EDA)

Univariate Analysis:

- Central tendency: mean, median, mode
- Dispersion: std dev, variance, range, IQR
- Distribution: histogram, box plot
- Skewness and kurtosis

Bivariate Analysis:

- Correlation analysis
- Scatter plots
- Crosstabs and contingency tables
- Covariance

Multivariate Analysis:

- Correlation matrices and heatmaps
- Pair plots
- Multidimensional relationships
- Dimensionality reduction

2.3 Statistics for Data Science

Probability Distributions:

Normal Distribution (Gaussian):

- Bell-shaped, symmetric
- 68-95-99.7 rule
- Assumed by many algorithms

Binomial Distribution:

- Success/failure outcomes
- Used for binary events
- A/B testing foundation

Poisson Distribution:

- Count data (events in time/space)
- Rare events
- Used in forecasting

Exponential Distribution:

- Time between events
- Memoryless property
- Reliability analysis

Hypothesis Testing:

- Null vs. alternative hypothesis
- Type I & Type II errors
- P-values and significance levels
- T-test, Z-test, Chi-square, ANOVA

Confidence Intervals:

- Range of plausible parameter values
- Interpretation: "95% confident parameter in range"
- Width depends on sample size and variability

Module 3: Machine Learning - Supervised Learning

3.1 Linear Regression

Model:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p + \epsilon$$

Assumptions (Gauss-Markov):

1. Linearity: True relationship is linear
2. Independence: Observations independent
3. Homoscedasticity: Constant error variance
4. Normality: Errors normally distributed
5. No multicollinearity: Predictors not perfectly correlated

Evaluation Metrics:

- R^2: Proportion of variance explained (0-1)
- RMSE: Root Mean Squared Error
- MAE: Mean Absolute Error
- Adjusted R^2: Accounts for number of predictors

When to Use:

- Continuous target variables
- Interpretability important
- Linear relationships
- Real estate pricing, demand forecasting

3.2 Logistic Regression

Model:

$$P(Y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X)}}$$

Key Features:

- Predicts probability (0-1)
- Linear in log-odds space

- Coefficients = log-odds multipliers
- e^{β_1} = odds ratio

Evaluation Metrics:

- Accuracy: Overall correctness
- Precision: TP / (TP + FP) - "When we predict yes, are we right?"
- Recall: TP / (TP + FN) - "Did we find all actual positives?"
- F1-Score: Harmonic mean of precision and recall
- AUC-ROC: Area under receiver operating characteristic

When to Use:

- Binary classification (0/1, yes/no)
- Probability estimates needed
- Interpretability important
- Default model to try first

3.3 Decision Trees & Ensemble Methods

Decision Trees:

- Split data recursively
- Easy to interpret
- Can overfit easily
- Prone to high variance

Random Forest:

- Ensemble of decision trees
- Bootstrap aggregating (bagging)
- Reduces overfitting
- Better generalization

Gradient Boosting (XGBoost, LightGBM):

- Sequential tree building
- Each tree corrects previous errors
- State-of-the-art performance
- Requires careful hyperparameter tuning

Why Ensemble Methods Work:

- Diversity in predictions
- Averaging reduces variance
- Correcting bias through boosting

3.4 Support Vector Machines (SVM)

Core Idea:

- Find optimal separating hyperplane
- Maximize margin between classes
- Handle non-linear boundaries with kernels

Kernel Types:

- Linear: For linearly separable data
- RBF: For non-linear boundaries
- Polynomial: For complex relationships

Advantages:

- Works well in high dimensions
- Effective with limited data
- Versatile (classification & regression)

Disadvantages:

- Computationally expensive for large datasets
- Less interpretable than trees
- Requires feature scaling

Module 4: Machine Learning - Unsupervised Learning

4.1 Clustering

K-Means Clustering:

- Divides data into k clusters
- Minimizes within-cluster variance
- Requires specifying k upfront
- Algorithm:

1. Initialize k centroids randomly
2. Assign points to nearest centroid
3. Update centroids
4. Repeat until convergence

Hierarchical Clustering:

- Creates tree-like structure (dendrogram)
- Agglomerative: bottom-up
- Divisive: top-down
- No need to specify cluster count upfront

DBSCAN:

- Density-based clustering
- Finds clusters of arbitrary shape
- Identifies outliers as noise
- Parameters: eps (radius), min_samples

Evaluation Metrics:

- Silhouette Score: -1 to 1 (higher = better separation)
- Davies-Bouldin Index: Lower is better
- Elbow method: Find optimal k

4.2 Dimensionality Reduction

Principal Component Analysis (PCA):

- Transforms data to uncorrelated components
- Preserves maximum variance
- Reduces computational cost
- Improves visualization

Steps:

1. Standardize features
2. Compute covariance matrix
3. Find eigenvectors/eigenvalues
4. Project data onto principal components

Applications:

- Visualization (PCA to 2D/3D)
- Noise reduction
- Feature extraction
- Preprocessing before ML

When to Use:

- Many correlated features
- Curse of dimensionality
- Need to visualize high-dimensional data

4.3 Recommendation Systems

Collaborative Filtering:

- User-based: Similar users → similar preferences
- Item-based: Similar items → similar ratings
- Matrix factorization: Latent factor models

Content-Based Filtering:

- Item features → recommendations
- User profile vs. item characteristics

Hybrid Approaches:

- Combine collaborative + content-based
- Often achieves best performance
- Used by Netflix, Amazon, Spotify

Module 5: Advanced Topics

5.1 Deep Learning Fundamentals

Neural Networks:

- Input → Hidden layers → Output
- Neurons and activation functions
- Forward propagation and backpropagation
- Learning through gradient descent

Activation Functions:

- ReLU: max(0, x) - Most common
- Sigmoid: 1 / (1 + e^-x) - Binary classification
- Tanh: Between -1 and 1
- Softmax: Multi-class classification

Architectures:

- Dense (Fully Connected)
- Convolutional (Images)
- Recurrent (Sequences)
- Transformers (NLP)

5.2 Natural Language Processing (NLP)

Text Preprocessing:

- Tokenization: Split into words
- Lowercase: Standardization
- Stemming/Lemmatization: Reduce to root
- Stop word removal
- TF-IDF: Term frequency-inverse document frequency

Word Embeddings:

- Word2Vec: Semantic relationships
- GloVe: Global vectors
- FastText: Subword information
- Captures meaning in vector space

Applications:

- Sentiment analysis
- Text classification
- Named entity recognition
- Machine translation

5.3 Computer Vision

Image Fundamentals:

- Pixels and color spaces
- Convolution operations

- Pooling layers
- Feature extraction

Deep Learning for Vision:

- CNN architectures: VGG, ResNet, Inception
- Transfer learning: Pre-trained models
- Object detection: YOLO, R-CNN
- Semantic segmentation

5.4 Model Evaluation & Validation

Cross-Validation:

- k-Fold: Divide into k subsets
- Stratified: Maintain class proportions
- Time-series: Respect temporal order
- Leave-One-Out: k = n observations

Metrics Selection:

- Imbalanced data: Use precision-recall, F1
- Business cost: Weight errors accordingly
- Domain-specific metrics

Overfitting vs. Underfitting:

- Overfitting: High train accuracy, low test accuracy
- Underfitting: Low accuracy on both
- Solutions: Regularization, more data, simpler model

Module 6: Real-World Case Studies

6.1 Amazon: Recommendation System

Challenge: Recommend products from millions of items to personalize experience

Solution: Collaborative filtering + matrix factorization

- Analyze purchase history
- Identify similar users

- Predict item ratings
- Rank recommendations

Results: 35% of sales from recommendations[2]

Key Insights:

- Scale matters (billions of data points)
- Real-time performance critical
- Continuous model retraining
- A/B testing for optimization

6.2 Netflix: Demand Forecasting

Challenge: Predict content demand for resource allocation

Solution: Time series + machine learning ensemble

- ARIMA for seasonal patterns
- Prophet for trend detection
- Machine learning for anomalies
- Ensemble averaging

Results: 98% forecast accuracy[2]

Key Insights:

- Multiple models often better than single
- External factors matter (events, holidays)
- Rolling forecasts improve accuracy
- Model monitoring essential

6.3 JPMorgan: Fraud Detection

Challenge: Real-time fraud detection without harming customer experience

Solution: Logistic regression + ensemble methods

- Transaction features engineering
- Behavioral profiling
- Real-time scoring
- Threshold optimization

Results: 40% fraud reduction, 0.5% false positive rate[2]

Key Insights:

- Business cost matters (Type I vs. II errors)
- Interpretability crucial for adoption
- Continuous model updates
- Feedback loops for improvement

6.4 Uber: Demand Prediction

Challenge: Forecast ride demand for surge pricing and driver allocation

Solution: Time series + spatial analysis

- Historical demand patterns
- Geographic clustering
- Event data integration
- Real-time adjustments

Results: 25% reduction in wait times[2]

Key Insights:

- External features critical
- Spatial dimension important
- Speed vs. accuracy tradeoff
- Deployed models must be fast

6.5 Google: Search Ranking

Challenge: Rank billions of web pages for query relevance

Solution: Learning-to-rank machine learning

- 200+ features per page
- Gradient boosting models
- Complex algorithms
- Continuous experimentation

Results: Billions of improved search results daily[2]

Key Insights:

- Feature engineering crucial
- Scale unprecedented
- A/B testing at massive scale
- Feedback loops from user behavior

Module 7: Practical Coding Examples

7.1 Data Loading & Exploration

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

Load data

df = pd.read_csv('data.csv')

Explore

print(df.head())
print(df.info())
print(df.describe())
print(df.isnull().sum())

Visualization

sns.pairplot(df)
sns.heatmap(df.corr(), annot=True)
plt.show()

7.2 Data Preprocessing

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
```

Handle missing values

```
imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(X)
```

Encode categorical

```
encoder = OneHotEncoder()
X_encoded = encoder.fit_transform(X_categorical)
```

Scale features

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_imputed)
```

7.3 Model Building & Evaluation

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report,
roc_auc_score
```

Split data

```
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y
)
```

Train models

```
lr = LogisticRegression()
rf = RandomForestClassifier(n_estimators=100)
```

```
lr.fit(X_train, y_train)
rf.fit(X_train, y_train)
```

Evaluate

```
y_pred_lr = lr.predict(X_test)
y_pred_rf = rf.predict(X_test)
```

```
print(classification_report(y_test, y_pred_lr))
print(f"AUC-ROC: {roc_auc_score(y_test, lr.predict_proba(X_test)[:,
1])}")
```

Cross-validation

```
scores = cross_val_score(rf, X_train, y_train, cv=5, scoring='roc_auc')
print(f"CV Scores: {scores.mean():.4f} (+/- {scores.std():.4f})")
```

7.4 Hyperparameter Tuning

```
from sklearn.model_selection import GridSearchCV,
RandomizedSearchCV
```

Grid Search

```
params = {
'n_estimators': [50, 100, 200],
'max_depth': [5, 10, 15],
'min_samples_split': [2, 5, 10]
}
```

```
grid_search = GridSearchCV(
RandomForestClassifier(),
params,
```

```
cv=5,
scoring='roc_auc',
n_jobs=-1
)
```

```
grid_search.fit(X_train, y_train)
print(f"Best params: {grid_search.best_params_}")
print(f"Best score: {grid_search.best_score_}")
```

7.5 Time Series Forecasting

```
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX
```

Fit ARIMA

```
model = ARIMA(time_series, order=(1, 1, 1))
results = model.fit()
```

Forecast

```
forecast = results.get_forecast(steps=30)
forecast_df = forecast.conf_int()
```

Visualization

```
plt.plot(time_series)
plt.plot(forecast_df.index, forecast_df.iloc[:, 0], color='red')
plt.fill_between(forecast_df.index,
forecast_df.iloc[:, 0],
forecast_df.iloc[:, 1],
alpha=0.3)
plt.show()
```

Module 8: Top Companies Interview Q&A

8.1 Fundamental Questions (Easy)

Q1: What is the difference between data science, machine learning, and artificial intelligence?

A:

- **Artificial Intelligence**: Broad field - machines performing tasks requiring intelligence
- **Machine Learning**: Subset of AI - systems learning from data without explicit programming
- **Data Science**: Interdisciplinary - extracting insights using ML, statistics, programming

Example hierarchy: AI ⊃ ML ⊃ Data Science

Q2: Explain the difference between supervised and unsupervised learning.

A:

- **Supervised**: Labeled data (input-output pairs), predict target variable
  - Classification: Discrete outputs
  - Regression: Continuous outputs
  - Examples: predicting house price, classifying emails
- **Unsupervised**: Unlabeled data, find patterns
  - Clustering: Group similar items
  - Dimensionality reduction: Reduce features
  - Examples: customer segmentation, anomaly detection

Q3: What is overfitting and how do you prevent it?

A: Overfitting = model learns training data too well, including noise

- Symptoms: High training accuracy, poor test accuracy
- Causes: Too complex model, insufficient data, high variance
- Solutions:
  - Use simpler model
  - Collect more data

- Regularization (L1, L2)
- Early stopping
- Cross-validation

Q4: Explain the bias-variance tradeoff.

A:

- **High Bias, Low Variance**: Underfitting, oversimplified model
- **Low Bias, High Variance**: Overfitting, overly complex model
- **Goal**: Balance both (low bias, low variance)
- Total error = $Bias^2$ + Variance + Irreducible error
- Increasing complexity decreases bias, increases variance

Q5: How do you handle imbalanced datasets?

A:

- **Resampling**: Oversample minority, undersample majority
- **SMOTE**: Synthetic minority oversampling
- **Class weights**: Penalize minority class errors more
- **Metrics**: Use precision-recall, F1, AUC-ROC (not accuracy)
- **Threshold adjustment**: Change decision boundary
- **Ensemble methods**: Combine predictions

Q6: Explain cross-validation and why it matters.

A:

- k-Fold: Divide data into k folds, train k-1, test on 1, repeat k times
- Advantages:
  - Uses all data for training and validation
  - Reduces variance in performance estimates
  - Better generalization assessment
- Time series: Maintain chronological order

Q7: What evaluation metrics would you use for classification?

A:

- **Accuracy**: (TP + TN) / Total - Overall correctness
- **Precision**: TP / (TP + FP) - When we predict positive, are we right?

- **Recall**: TP / (TP + FN) - Did we find all actual positives?
- **F1-Score**: Harmonic mean of precision and recall
- **AUC-ROC**: Threshold-independent metric
- Choose based on business impact of errors

Q8: How would you diagnose and fix high bias?

A:

- **Symptoms**: Low training and test accuracy
- **Causes**: Model too simple, insufficient features
- **Fixes**:
  - More complex model
  - Feature engineering
  - Reduce regularization
  - More training iterations

Q9: Explain feature engineering and its importance.

A: Creating new features from raw data to improve model performance

- **Techniques**:
  - Scaling/normalization
  - Polynomial features
  - Interaction terms
  - Domain knowledge features
- **Importance**: Often more impactful than algorithm choice
- **Examples**: Creating "age group" from age, "day of week" from date

Q10: What is the curse of dimensionality?

A: Performance degradation with too many features

- **Problems**:
  - Sparse data
  - Increased computational cost
  - Overfitting risk
  - Difficulty in visualization
- **Solutions**: PCA, feature selection, domain knowledge

8.2 Intermediate Questions

Q11: Explain regularization (L1, L2) and when to use each.

A:

- **L1 (Lasso)**: $\lambda \sum |\beta_j|$
  - Shrinks coefficients exactly to zero
  - Feature selection
  - Sparse solutions
- **L2 (Ridge)**: $\lambda \sum \beta_j^2$
  - Shrinks but doesn't eliminate
  - Handles multicollinearity
  - Smooth solutions
- **Elastic Net**: Combination of L1 and L2
- Use L1 for feature selection, L2 for multicollinearity

Q12: How do you approach a classification problem with imbalanced data?

A:

1. **Understanding**: Analyze class distribution
2. **Resampling**:
   - Oversample minority (SMOTE)
   - Undersample majority
   - Combine both (SMOTEENN)
3. **Algorithm choice**: Use tree-based models (robust to imbalance)
4. **Metrics**: Precision-recall, F1-score, AUC-ROC
5. **Threshold**: Optimize decision boundary
6. **Ensemble**: Combine multiple approaches

Q13: Explain ensemble methods and their advantage.

A: Combining predictions from multiple models

- **Bagging**: Bootstrap aggregating, reduce variance
  - Random Forest, Extra Trees
- **Boosting**: Sequential, each corrects previous errors
  - AdaBoost, Gradient Boosting, XGBoost
- **Stacking**: Meta-learner combines base learners
- **Advantages**:

    - Better generalization
    - Reduced overfitting
    - Leverages strengths of multiple models

Q14: How do you select between different models?

A:

1. **Problem type**: Classification vs. regression vs. clustering
2. **Data characteristics**: Size, dimensionality, linearity
3. **Interpretability needs**: Business requirements
4. **Computational constraints**: Speed and resources
5. **Cross-validation**: Compare performance
6. **Business metrics**: Align with objectives
7. **Complexity**: Prefer simpler models (Occam's razor)

Q15: Explain A/B testing and how to design an experiment.

A:

1. **Hypothesis**: Null (no difference) vs. alternative
2. **Metrics**: Define KPI (conversion rate, revenue, etc.)
3. **Sample size**: Power analysis (typically 80% power)
4. **Duration**: Account for seasonality and cycles
5. **Randomization**: Random assignment to control/treatment
6. **Analysis**: Statistical test (t-test for differences)
7. **Multiple testing**: Correct for multiple comparisons
8. **Business significance**: Beyond statistical significance

Q16: How do you handle multicollinearity?

A:

- **Detection**: VIF > 5-10, high correlation matrix values
- **Problems**:
    - Inflated standard errors
    - Unstable coefficients
    - Poor generalization
- **Solutions**:
    - Drop one variable from correlated pair
    - Combine variables
    - Regularization (L2)

    - PCA for dimensionality reduction

Q17: Explain the difference between generalization and overfitting.

A:

- **Generalization**: Model performs well on unseen data
- **Overfitting**: Model performs well on training but poorly on test
- **Goal**: Find balance in bias-variance tradeoff
- **Assessment**: Cross-validation, test set performance
- **Signs of overfitting**: Large gap between train and test accuracy

Q18: How do you approach missing data?

A:

1. **Analyze**: Check pattern (MCAR, MAR, MNAR)
2. **Percentage**: If >50%, may need external data
3. **Methods**:
    - Deletion: If small proportion and random
    - Mean/median: Simple but biased
    - KNN: Preserve relationships
    - Model-based: Predictive imputation
4. **Validation**: Compare imputation methods' impact

Q19: Explain feature importance and its interpretation.

A: Measures how much each feature contributes to predictions

- **Methods**:
    - Tree-based: Gini importance
    - Linear: Coefficient magnitude (after standardization)
    - Permutation: Performance drop when feature shuffled
    - SHAP: Game theory approach
- **Uses**:
    - Identify most valuable features
    - Domain knowledge validation
    - Dimensionality reduction
    - Model interpretability

Q20: What is the difference between correlation and causation?

A:

- **Correlation**: Variables move together
- **Causation**: One causes the other
- **Risk**: Confounding variables
- **Example**: Ice cream and drowning correlate (both increase in summer)
- **Testing**: Controlled experiments better than observational

8.3 Advanced Questions

Q21: Design a recommendation system for an e-commerce platform.

A:

1. **Problem understanding**: Items to recommend for users
2. **Data collection**: User behavior, item features, interactions
3. **Approaches**:
    - Collaborative filtering: User-user, item-item
    - Content-based: Item features → similarity
    - Hybrid: Combine approaches
4. **Implementation**:
    - Build user-item interaction matrix
    - Matrix factorization (SVD)
    - Nearest neighbors search
5. **Evaluation**:
    - Precision@k, Recall@k
    - NDCG (Normalized Discounted Cumulative Gain)
    - Business metrics: Click-through rate, conversion
6. **Deployment**: Real-time serving, cold start handling

Q22: How would you build a fraud detection system?

A:

1. **Feature engineering**: Transaction, user, temporal features
    - Amount, velocity, geography, device, merchant
2. **Data imbalance**: Fraud rare (~0.1%)
    - SMOTE, class weights, threshold optimization
3. **Models**:

- Logistic regression: Interpretable
   - Random Forest: Captures interactions
   - Gradient boosting: State-of-the-art
   - Ensemble: Best performance
4. **Real-time requirements**:
   - Model latency < 100ms
   - Batch preprocessing
   - Efficient data structures
5. **Monitoring**:
   - Model performance drift
   - False alarm rate (business cost)
   - Feedback loops from investigators

Q23: Design a churn prediction model.

A:

1. **Target definition**: Customers not active in last N months
2. **Features**:
   - Behavioral: Usage frequency, duration, spending
   - Account: Tenure, value, plan type
   - Engagement: Support tickets, feature usage
3. **Imbalance handling**: Typically 10-30% churn
4. **Model selection**:
   - Logistic regression: Interpretable
   - XGBoost: High performance
5. **Evaluation**:
   - Recall important: Catch at-risk customers
   - Precision: Avoid wasting resources
   - Business cost analysis
6. **Deployment**:
   - Probability scores for intervention prioritization
   - Regular retraining (monthly/quarterly)
7. **Action**: Target interventions, retention offers

Q24: Explain how to optimize a recommendation system.

A:

1. **Offline evaluation**:
   - Precision@k, Recall@k, NDCG

    - Diversity metrics
    - Cold start performance
2. **Online evaluation** (A/B tests):
    - Click-through rate
    - Conversion rate
    - Revenue per user
    - User engagement
3. **Improvements**:
    - Feature engineering
    - Model ensembles
    - Contextual recommendations
    - Real-time updates
4. **Challenges**:
    - Cold start (new users/items)
    - Exploration vs. exploitation
    - Computational efficiency
    - Feedback loops and bias

Q25: Design end-to-end machine learning pipeline.

A:

1. **Data collection**: APIs, databases, logs
2. **Data preprocessing**:
    - Cleaning
    - Feature engineering
    - Scaling
3. **Model development**:
    - Algorithm selection
    - Hyperparameter tuning
    - Cross-validation
4. **Evaluation**:
    - Offline metrics
    - Cross-validation
    - Test set performance
5. **Deployment**:
    - Model serialization
    - API wrapper
    - Containerization (Docker)
    - Serving (Flask, FastAPI)

6. **Monitoring**:
    - Performance metrics
    - Data drift detection
    - Model retraining triggers
7. **Iteration**:
    - A/B testing
    - Feedback collection
    - Continuous improvement

Module 9: Practice Questionnaire

9.1 Basic Level (Beginner)

1. **What are the main steps in a data science project?**
2. **Explain the difference between training and test datasets.**
3. **What is a feature and why is feature engineering important?**
4. **Define overfitting and give an example.**
5. **What is the purpose of cross-validation?**
6. **Explain the difference between classification and regression.**
7. **What are the main challenges in machine learning?**
8. **Define accuracy and explain when it's misleading.**
9. **What is regularization and why use it?**
10. **Explain the bias-variance tradeoff.**

9.2 Intermediate Level

11. **How do you evaluate a classification model with imbalanced data?**
12. **Explain ensemble methods and their advantages.**
13. **What is feature selection and why is it important?**
14. **How do you handle missing data? List at least 5 methods.**
15. **Explain the difference between L1 and L2 regularization.**
16. **How would you approach a time series forecasting problem?**
17. **What is hyperparameter tuning? Describe two methods.**
18. **Explain matrix factorization in recommendation systems.**
19. **How do you diagnose and fix high variance?**
20. **What is a confusion matrix and what does each element represent?**

9.3 Advanced Level

21. **Design a customer churn prediction system from scratch.**
22. **Explain how to build a production-ready ML pipeline.**
23. **How would you approach a new problem with limited data?**
24. **Describe how to implement transfer learning for a new task.**
25. **Design a system to detect anomalies in real-time data.**

9.4 Case Study Questions

26. **Review Amazon recommendation case: How would you evaluate success?**
27. **Netflix forecasting: How do you handle new shows/special events?**
28. **JPMorgan fraud detection: Design the feature engineering process.**
29. **Uber demand prediction: How do you incorporate external factors?**
30. **Design a machine learning solution for a problem in your domain.**

9.5 Coding Questions

31. **Implement k-means clustering from scratch.**
32. **Write code to perform hyperparameter tuning using GridSearchCV.**
33. **Build a logistic regression classifier with cross-validation.**
34. **Implement feature scaling and encoding pipeline.**
35. **Write code to detect outliers using IQR method.**

Conclusion

The journey from beginner to advanced data scientist requires:

1. **Strong Fundamentals**: Master statistics, programming, and mathematics
2. **Continuous Learning**: Stay current with new algorithms and tools
3. **Practical Experience**: Build real projects with actual data
4. **Business Understanding**: Connect ML to business problems

5. **Communication Skills**: Explain complex findings clearly
6. **Ethical Awareness**: Responsible AI and fairness
7. **Problem-Solving**: Approach challenges systematically

Success in data science comes from combining technical skills with business acumen and domain expertise. Focus on understanding problems deeply, engineering features thoughtfully, selecting appropriate algorithms, and validating results rigorously.

References

[1] McKinsey & Company. (2025). The state of AI in 2025. Retrieved from https://www.mckinsey.com/capabilities/mckinsey-digital/our-insights/state-of-ai-2025

[2] Turing. (2025). 10 Real-World Data Science Case Studies. Retrieved from https://www.turing.com/resources/data-science-case-studies

[3] Coursera. (2025). Data Science Learning Roadmap. Retrieved from https://www.coursera.org/resources/data-science-learning-roadmap

[4] DataCamp. (2025). Machine Learning with Python. Retrieved from https://www.datacamp.com/tutorial/machine-learning-python

[5] Scikit-learn Documentation. (2025). Machine learning in Python. Retrieved from https://scikit-learn.org/

[6] TensorFlow. (2025). Deep Learning Framework. Retrieved from https://www.tensorflow.org/

[7] PyTorch. (2025). Deep Learning Framework. Retrieved from https://pytorch.org/

Document Version: 1.0
Last Updated: February 22, 2026
Author: Siddharth Vidyarthi

This comprehensive guide covers Data Science from foundational concepts through advanced techniques, featuring real-world case studies from FAANG companies, practical Python code examples, top company interview questions, and hands-on exercises suitable for aspiring data scientists and professionals seeking career advancement.